第4章 第11节 结构化输出与常用模板

- 来源: https://class.imooc.com/lesson/5270#mid=104912
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第4周 理解大模型开发
- 模块: Prompt基础与必要场景
- 课节: 图文 1-3 结构化输出与常用模板

1-3 结构化输出与常用模板

第4章 第11节 结构化输出与常用模板

> [!TIP] 阅读指南 上一节学习了Prompt五要素。现在进一步——如何让AI输出精确的JSON格式以便程序处理？有哪些现成的Prompt模板可以直接参考？本节给出答案。

11.1 结构化输出

强制AI按照特定的数据结构输出，便于程序处理。

方法1 只在Prompt中要求AI返回JSON

```
# 仅通过Prompt要求，AI输出JSON格式的回答
response = client.chat.completions.create(
    model="qwen3.6-plus"
    messages=[
        {
            "role": "system",
            "content": """请分析代码并返回JSON格式的结果。

输出格式示例：
{
  "complexity": "高/中/低",
  "issues": [...],
  "suggestions": [...]
}
"""
        },
        {"role": "user", "content": "分析这段代码：[your code]"}
    ]
)

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19

预览
```

AI实际可能输出：

```
"好的，我来分析一下这段代码：

`\`\`json

{"complexity": "中", ...}

`\`\`

从分析结果看，代码质量还不错。.."
```

这里的问题是什么?

问题在于，其实只需要那段JSON数据（{ }内的内容），而不需要AI的那些"自言自语"。但AI很难精确控制。这个时候，需要手动清洗，提取出JSON数据：

```
import json
content = response.choices[0].message.content
if "`\`\`json" in content:
    content = content.split("`\`\`json")[1].split("`\`\`")[0]
result = json.loads(content.strip())  # 可能报错!
```

方法2 使用API强制JSON模式

用API参数强制约束，AI无法"自由发挥"

```
response = client.chat.completions.create(
    model="qwen3.6-plus"
    messages=[
        {
            "role": "system",
            "content": """请分析以下代码，以JSON格式返回结果：

{
  "complexity": "复杂度等级（高/中/低）",
  "issues": [...],
  "suggestions": [...]
}
"""
        },
        {"role": "user", "content": "分析这段代码：[your code]"}
    ],
    response_format={"type": "json_object"}  # 关键区别在这里！
)

# AI实际输出：
# {"complexity": "中", "issues": [...], "suggestions": [...]}
# 纯JSON，没有任何额外文字！

# 直接解析，不需要任何清洗
import json
result = json.loads(response.choices[0].message.content)  # 100%成功！
print(result["complexity"])
print(result["issues"])
```

两种方法的本质区别

这两种方法看起来差不多——都是让AI输出JSON，到底有什么不同？

关键在于约束力度。

约束力度

方法1就像口头跟AI说"请给我JSON格式"。

方法1

AI大部分时候会照做，但它可能会"热心"地额外多给一些东西——比如在JSON前面加一句"好的，分析结果如下："，或者用json代码块把JSON包起来，甚至在后面再解释一遍。这些"好心"在人看来挺友好，但对程序来说就是灾难：json.loads()一遇到这些额外的文字就报错。

需要手动写代码去"清洗"这些输出：去掉前面的说明文字、去掉代码块标记、去掉后面的解释……这在批量处理时特别麻烦，因为AI每次"热心"的方式还不太一样，清洗逻辑很难覆盖所有情况。

方法2就不一样了。

方法2

它通过API的response_format={"type": "json_object"}参数，在底层直接锁死了输出格式——AI想"热心"都不行，它只能输出纯粹的JSON字符串，一个多余的字都不会有。这就像给AI套上了一个严格的模板，保证输出的东西可以直接喂给json.loads()，不需要任何预处理。

使用场景选择

如果是在写一个工具、做批量处理、或者构建生产系统，那必须用方法2。稳定性是第一位的，不能让程序因为解析失败就崩掉。

但如果只是临时用一下，或者希望AI在给出JSON的同时还能解释一下结果（比如给人看的报告），那方法1反而更合适——它允许AI"说人话"。

简单说：机器读的用方法2，人读的用方法1。

机器读的用方法2，人读的用方法1

11.2 常用Prompt模板

学完了Prompt设计的五要素，自然会想到一个问题：有没有现成的模板可以直接拿来用？

确实有现成的模板，但别照搬。

下面这些模板是我在实际项目中总结出来的，覆盖了大部分常见场景。但记住，它们只是启发性的参考——你的业务场景、数据特点、输出要求都不一样，照抄模板往往效果不佳。真正好用的Prompt，都是在这些模板基础上，根据具体情况调整出来的。

通用五要素模板

这是最基础的结构，适合任何场景：

```
BASE_PROMPT = """你是{role}。

任务：{task}

背景信息：
{context}

输出要求：
{format}

约束条件：
{constraints}
"""
```

使用示例

```
prompt = BASE_PROMPT.format(
    role="Python后端工程师，熟悉FastAPI和SQLAlchemy",
    task="审查这段API代码，找出性能问题",
    context="这是一个用户查询接口，日访问量10万次",
    format="列出问题清单，每项包含：行号、问题、影响、建议",
    constraints="只分析性能，不关注代码风格"
)
```

代码生成模板

适合让AI写代码的场景：

```
CODE_GEN_PROMPT = """你是{language}开发专家。

需求：{requirement}

技术约束：
- 语言/框架：{tech_stack}
- 代码风格：{style}

输出格式：
1. 完整可运行的代码
2. 关键部分的注释
3. 简单的使用示例

注意事项：
- 包含异常处理
- 遵循最佳实践
- 不使用废弃API
"""
```

```
prompt = CODE_GEN_PROMPT.format(
    language="Python",
    requirement="实现一个异步的Redis缓存装饰器",
    tech_stack="aioredis 2.0",
    style="符合PEP 8规范"
)
```

实际调整示例 如果你的项目有特殊要求，可以这样调整：

实际调整示例

```
prompt = """你是Python开发专家，熟悉异步编程。

需求：实现一个异步的Redis缓存装饰器

技术约束：
- 使用aioredis 2.0
- 支持设置过期时间
- 缓存key自动生成（函数名+参数hash）

输出格式：
1. 装饰器完整代码
2. 使用示例（包含FastAPI路由）

注意事项：
- 必须处理Redis连接失败的情况
- 参数序列化要支持复杂对象
- 添加日志记录缓存命中率
"""
# 根据实际需求加了很多细节
```

文本处理模板

摘要、改写、提取等场景：

```
TEXT_PROCESS_PROMPT = """你是{role}。

任务：{task}

输入文本的特点：
{text_info}

输出要求：
- 长度：{length}
- 风格：{style}
- 受众：{audience}

约束：
- 不添加原文没有的信息
- 保持客观中立
- {specific_constraint}

文本：
{text}
"""
```

```
# 场景1：技术文档摘要
prompt1 = TEXT_PROCESS_PROMPT.format(
    role="技术文档编辑",
    task="将这篇API文档总结为200字的开发者指南",
    text_info="官方API文档，包含大量技术细节",
    length="200字左右",
    style="简洁、技术化",
    audience="有经验的开发者",
    specific_constraint="保留所有代码示例的关键部分",
    text="[文档内容]"
)
```

细心的读者可能已经注意到了：当 Prompt 比较复杂时，大多数情况下我们都会用"Markdown"的语法来组织它——用标题分区、用有序或无序列表列要求、用代码块包示例。

这样组织有三个好处：

- 模型更容易理解。模型在训练时看过海量的 Markdown 文档（技术文档、GitHub README、代码注释），对这种结构非常熟悉，能快速识别重点并按结构生成内容。
- 边界更清晰。用代码块包住示例，模型就知道"这是例子"；用列表列出约束，模型就知道"这些必须满足"。歧义少了，输出自然更稳定。

- 协作更高效。别人一看就懂哪些是硬性规则、哪些是风格偏好，修改和复用都方便。

11.3 下一节预告

现在已经掌握了大模型应用开发的基础：API调用、参数控制、Prompt设计。但在实际应用中，可能会遇到这样的问题：AI的回答总是基于训练数据，如何让它回答我们自己的私有知识？比如公司内部文档、项目代码库、业务规则？

下一章将学习 RAG（检索增强生成），让AI能够访问专属知识库，回答它原本"不知道"的问题。

11.4 ■ 学点英语

中文	English	音标	说明
结构化输出	Structured Output	/ˈstrʌktʃərd ˈaʊtpʊt/	让AI按特定数据结构（如JSON）返回结果
JSON模式	JSON Mode	/ˈdʒeɪsɑːn moʊd/	通过response_format参数强制AI输出纯JSON
响应格式	Response Format	/rɪˈspɑːns ˈfɔːrmæt/	API参数，指定AI返回内容的格式要求
提示词模板	Prompt Template	/prɑːmpt ˈtemplət/	可复用的提示词结构框架
Markdown语法	Markdown Syntax	/ˈmɑːrkdaʊn ˈsɪntæks/	用于组织复杂Prompt的轻量级标记语言